

Microsoft Sentinel RDP Brute-Force Detection Lab

Rebuilding an RDP Intrusion-Detection Lab on a Cloud-Native SIEM: Data Connectors, KQL Hunting, a Real Triggered Incident, and Automated Response

Hakim Fakhar

1 Introduction

I'm Hakim Fakhar, a Cybersecurity Engineering student at ENSA (Cybersecurity & Embedded Systems track), working toward a SOC analyst role. I built this lab to take an attack I had already detected once – an RDP brute-force against a Windows box, originally caught with Wazuh, Sysmon, Suricata, and Chainsaw – and rebuild the entire detection side of it natively in Microsoft Sentinel.

Same target VM, same attacker tooling (Hydra), completely different pipeline: Azure Monitor Agent and Sysmon feeding Log Analytics, KQL hunting queries written by hand before any rule existed, a scheduled Analytics Rule built from that hunting logic, a real Incident that it actually triggered, and a Logic App playbook that closes the loop with an automatic notification. This report follows the same approach I used for the original Wazuh lab and for a separate AWS IAM compromise lab: build it, break it, hunt through the logs by hand first, and write down honestly what worked and what I had to fix along the way – including a detection-logic mistake that mattered as much as anything that went right.

Contents

1	Introduction	1
2	Lab Architecture	1
2.1	Environment	1
2.2	Detection Stack	1
2.3	Setup and Verification	2
3	Data Connectors – AMA and Sysmon	2
4	Attack Chain	5
4.1	Hydra RDP Brute-Force	5
4.2	Recon	6
5	Detection Engineering – KQL Hunting	8
6	Analytics Rule and Triggered Incident	11
7	Incident Investigation	12
8	Workbook	13

9 MITRE ATT&CK Mapping	16
10 Automation – SOAR Playbook	17
11 Lessons Learned	21
12 Recommendations	21

2 Lab Architecture

2.1 Environment

- **Target:** the same Windows 10 VM (VirtualBox, hostname DESKTOP-8C93D80) used in the original Wazuh RDP lab, onboarded to Azure via **Azure Arc** rather than deployed as a fresh Azure VM – keeping this project at effectively zero cost, the same reasoning I applied after the billing scare in the AWS IAM lab.
- **Workspace:** Log Analytics workspace `law-sentinel-lab`, resource group `rg-sentinel-lab`, North Europe (West Europe rejected new accounts at the time).
- **Attacker:** Kali Linux, Hydra v9.6, running against the target's RDP service over the lab's local network.

2.2 Detection Stack

- **Azure Monitor Agent (AMA)** – the direct equivalent of the Wazuh agent in the original lab, installed automatically once the VM was onboarded through the *Windows Security Events* via *AMA* content-hub solution.
- **Two Data Collection Rules** feeding `law-sentinel-lab` from the same Arc-onboarded machine: `dcr-windows-security-sysmon-lab` for the Windows Security event log (logon events), and `dcr-sysmon-events-lab` for the Sysmon operational log (process creation).
- **Microsoft Sentinel** on top of the workspace – Analytics Rules, Incidents, Workbooks, the MITRE ATT&CK view, and Automation all live here.

2.3 Setup and Verification

Create Log Analytics workspace ...

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group *
[Create new](#)

Instance details

Name *

Region *

[Review + Create](#) [Previous](#) [Next: Tags >](#)

Figure 1: Creating the Log Analytics workspace `law-sentinel-lab`.

Didn't find what you were looking for? We're showing a limited set of results. Try refining your search for more specific results. [Learn more](#)

Search: windows security events

Status: All Content type: All Support: All Provider: All Category: All Content sources: All

Content title	Status	Content source	Provider
☒ Windows Security Events	☐ Not installed	Solution	Microsoft
Windows Security Events via AMA	☐ Not installed	Solution	Microsoft
Security Events via Legacy Agent	☐ Not installed	Solution	Microsoft
NRT Security Event log cleared	☐ Not installed	Solution	Microsoft
Suspicious Windows Login Outside Normal...	☐ Not installed	Solution	Microsoft
Windows System Time changed on hosts	☐ Not installed	Solution	Microsoft
Windows System Shutdown (RobustGuardian)	☐ Not installed	Solution	Microsoft

Windows Security Events via AMA

Disconnected Status Microsoft Provider Last Log Receiv...

Description
You can stream all security events from the Windows machines connected to your Microsoft Sentinel workspace using the Windows agent. This connection enables you to view dashboards, create custom alerts, and improve investigation. This gives you more insight into your organization's network and improves your security operation capabilities. For more information, see the [Microsoft Sentinel documentation](#).

Last data received
--

Content source Version

[Install Solution](#)

Figure 2: Installing the *Windows Security Events* solution from the Sentinel content hub – this is what unlocks the AMA-based connector used below.

3 Data Connectors – AMA and Sysmon

Two DCRs route telemetry from the Arc-onboarded VM to the workspace: one for the standard Windows Security log, one for Sysmon.

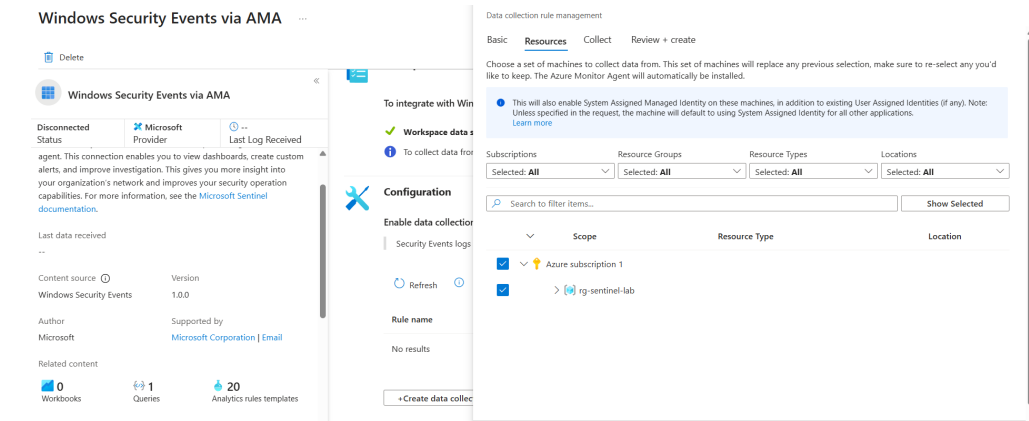


Figure 3: Selecting desktop-8c93d80 (Azure Arc machine) as the DCR resource.

Basic	
Data rule name	dcr-windows-security-sysmon-lab
Subscription	Azure subscription 1
Resource Group	rg-sentinel-lab
Selected resources	
Name	Type
desktop-8c93d80	microsoft.hybridcompute/machines
Selected events	
Common	

Figure 4: dcr-windows-security-sysmon-lab – Security Events log, common event set.

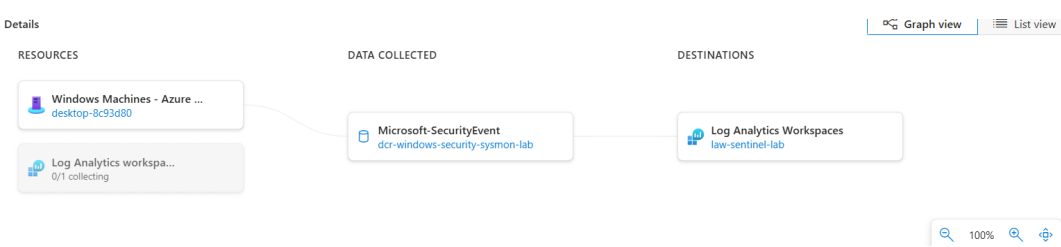


Figure 5: First DCR wired end to end: resource → Microsoft-SecurityEvent → workspace.

Basics	
Rule Name	dcr-sysmon-events-lab
Subscription	Azure subscription 1
Resource Group	rg-sentinel-lab
Region	North Europe
Type of telemetry	Agent-based - Windows
Data collection endpoint	<none>
Resources	
DESKTOP-8C93D80	Machine - Azure Arc
Collect and deliver	
Windows Event Logs	Log Analytics Workspaces (law-sentinel-lab)
Tags	

Figure 6: `dcr-sysmon-events-lab` – Sysmon via Windows Event Logs, agent-based telemetry.

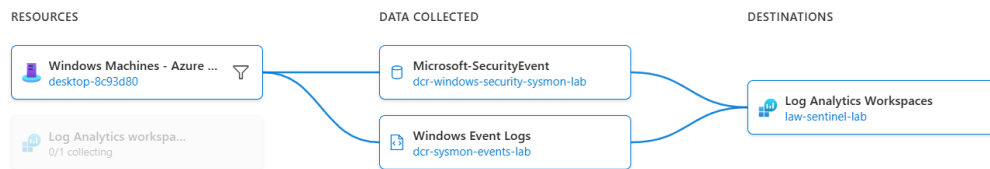


Figure 7: Both DCRs feeding `law-sentinel-lab` from the same resource – Security Events and Sysmon side by side.

4 Attack Chain

I re-ran the same attack sequence used against this VM in the original Wazuh lab: a remote Hydra brute-force against RDP first, then local recon once a valid credential got me on the box.

4.1 Hydra RDP Brute-Force

The first dictionary run against `Users.txt` / `passwords.txt` never got a clean answer either way – FreeRDP’s connection kept failing before a login attempt could even complete, which the RDP module’s own warning banner half-admits (“the rdp module is experimental”).

```
(sylix@kali)-[~]
$ hydra -L Users.txt -P passwords.txt rdp://192.168.1.49
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is
non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-06 13:30:44
[WARNING] rdp servers often don't like many connections, use -t 1 or -t 4 to reduce the number of parallel connections and -W 1 or -W 3 to wait between
connection to allow the server to recover
[INFO] Reduced number of tasks to 4 (rdp does not like many parallel connections)
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 4 tasks per 1 server, overall 4 tasks, 16 login tries (l:4/p:4), ~4 tries per task
[DATA] attacking rdp://192.168.1.49:3389/
[3389][rdp] account on 192.168.1.49 might be valid but account not active for remote desktop: log [REDACTED], continuing attacking th
e account.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-06 13:30:52
```

Figure 8: First Hydra run – 16 login tries, every attempt failing at the connection layer rather than the credential-check layer.

I re-ran the attack again days later to see whether the same noisy-but-inconclusive pattern held up, and it did – same connection failures, same zero-valid-password result.

```
(sylix@kali)-[~]
$ hydra -L Users.txt -P passwords.txt rdp://192.168.1.49
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is
non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-07 15:14:01
[WARNING] rdp servers often don't like many connections, use -t 1 or -t 4 to reduce the number of parallel connections and -W 1 or -W 3 to wait between
connection to allow the server to recover
[INFO] Reduced number of tasks to 4 (rdp does not like many parallel connections)
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 4 tasks per 1 server, overall 4 tasks, 20 login tries (l:5/p:4), ~5 tries per task
[DATA] attacking rdp://192.168.1.49:3389/
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-07 15:14:18
```

Figure 9: Second dictionary run, several days later – consistent with the first: connection failures, no valid password recovered.

A targeted, single-credential attempt is what actually got in.

Finding: the failed dictionary runs are exactly what the detection is built to catch

Neither Hydra run against `Users.txt`/`passwords.txt` ever completed a full RDP session – they never got past the connection layer. But every one of those attempts still generated a **failed network logon (EventID 4625, LogonType 3)** on the target, because Windows logs the authentication attempt regardless of whether the RDP session itself ever fully established. That is precisely the signal the brute-force detection in Section 5 is built around: it doesn’t

```
(sylix@kali)-[~]
└─$ hydra -i sylix -p Mrhakim12 rdp://192.168.1.49
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is
non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-06 13:33:03
[WARNING] rdp servers often don't like many connections, use -t 1 or -t 4 to reduce the number of parallel connections and -W 1 or -W 3 to wait between
connection to allow the server to recover
[INFO] Reduced number of tasks to 4 (rdp does not like many parallel connections)
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking rdp://192.168.1.49:3389/
[3389][rdp] host: 192.168.1.49 login: password:
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-06 13:33:03
```

Figure 10: A single targeted username/password guess succeeds where the dictionary runs didn't.

need a completed session, only a burst of failed network logons from one source in a short window. The single successful, targeted credential guess is what finally produced a matching **successful** logon (EventID 4624, LogonType 10) alongside that burst – the two together are what make this a believable brute-force-then-compromise story rather than just noisy failed scans.

4.2 Recon

Once on the box, I ran the same recon commands as the original Wazuh lab.

```
PS C:\Users\sylix> whoami
desktop-8c93d80\sylix
PS C:\Users\sylix> systeminfo

Host Name:                DESKTOP-8C93D80
OS Name:                   Microsoft Windows 10 Pro
OS Version:                10.0.19045 N/A Build 19045
OS Manufacturer:          Microsoft Corporation
OS Configuration:         Standalone Workstation
OS Build Type:              Multiprocessor Free
Registered Owner:          sylix
Registered Organization:
Product ID:                 00330-80000-00000-AA186
Original Install Date:      6/19/2026, 5:29:37 AM
System Boot Time:           9/6/2026, 1:14:23 PM
System Manufacturer:        innotek GmbH
System Model:               VirtualBox
System Type:                x64-based PC
Processor(s):               1 Processor(s) Installed.
[01]: Intel64 Family 6 Model 154 Stepping 3 GenuineIntel ~2688 Mhz
BIOS Version:               innotek GmbH VirtualBox, 12/1/2006
Windows Directory:          C:\Windows
System Directory:           C:\Windows\system32
Boot Device:                \Device\HarddiskVolume1
System Locale:               en-us;English (United States)
```

Figure 11: whoami and systeminfo on the target.

```

S C:\Users\sylix> net user

User accounts for \\DESKTOP-8C93D80

-----
Administrator          DefaultAccount          Guest
sylix                  WDAGUtilityAccount
The command completed successfully.

S C:\Users\sylix> net localgroup

Aliases for \\DESKTOP-8C93D80

-----
Access Control Assistance Operators
Administrators
Backup Operators
Cryptographic Operators
Device Owners
Distributed COM Users
Event Log Readers
Guests
Hybrid agent extension applications
Hyper-V Administrators
IIS_IUSRS
Network Configuration Operators
Performance Log Users
Performance Monitor Users
Power Users
Remote Desktop Users
Remote Management Users
Replicator
System Managed Accounts Group
Users
The command completed successfully.

```

Figure 12: net user / net localgroup – local account and group enumeration.

```

PS C:\Users\sylix> Get-Process

Handles  NPM(K)  PM(K)  WS(K)  CPU(s)  Id  SI ProcessName
-----
116      7       3984   7932           4452  0  AggregatorHost
191     13      2708  12824           3408  0  AMAExtHealthMonitor
420     25     11932  31500         0.53  1064  1  ApplicationFrameHost
160     12     15196  12144           3020  0  arcproxy
453     35     11004   1336         0.16  616  1  CalculatorApp
273     14      5956  20520         1.19  3312  1  conhost
112      7      6240   6056           6024  0  conhost
108      7      6220   6072           6504  0  conhost
595     17      1944   4964           452  0  csrss
395     15      1824   5232           528  1  csrss
174     12      1664   5600           3576  3  csrss
425     16      4036  20512         0.38  5052  1  ctfmon
211     17      3940  10564           3160  0  dllhost
246     23      5080  13160         0.23  3464  1  dllhost
1041    50     40284  82020           404  1  dwm
774     24     16992  43532           4060  3  dwm
2275    92     64968  151520        12.69  4512  1  explorer
50      6      1476   3876           772  0  fontdrvhost
50      7      1692   4408           780  1  fontdrvhost
50      6      1412   5072           8152  3  fontdrvhost
339     20      5944  22000           5744  0  gc_arc_service
375     21      6324  23604           1784  0  gc_extension_service
424     23     25788  31968           4188  0  himds
0       0        60     8              0  0  Idle
646     36     15228  53220           5492  3  LogonUI
1475    25      7708  18572           680  0  lsass
0       0        420   39476           1828  0  Memory Compression
282     19     10124  19296           4264  0  MetricsExtension.Native

```

Figure 13: Get-Process – process enumeration.

5 Detection Engineering – KQL Hunting

Before writing any scheduled rule, I hunted manually against `SecurityEvent` and the `Sysmon Event` table, working out the exact filters that separate signal from noise.

The screenshot shows a KQL query in the Microsoft Sentinel interface. The query is: `SecurityEvent | where EventID == 4625 | project TimeGenerated, IPAddress, LogonType, TargetUserName | order by TimeGenerated desc | take 20`. The results table shows 10 rows of failed logon attempts from 9/6/2026.

TimeGenerated [UTC]	IPAddress	LogonType	TargetUserName
> 9/6/2026, 12:31:41.032 PM	192.168.1.56	3	sylix
> 9/6/2026, 12:31:26.444 PM	192.168.1.56	3	sylix
> 9/6/2026, 12:31:26.443 PM	192.168.1.56	3	sylix
> 9/6/2026, 12:31:26.440 PM	192.168.1.56	3	sylix
> 9/6/2026, 12:31:00.503 PM	192.168.1.56	3	kalipo
> 9/6/2026, 12:31:00.088 PM	192.168.1.56	3	kalipo
> 9/6/2026, 12:31:00.064 PM	192.168.1.56	3	kalipo
> 9/6/2026, 12:30:59.996 PM	192.168.1.56	3	kalipo
> 9/6/2026, 12:30:58.733 PM	192.168.1.56	3	ginomo
> 9/6/2026, 12:30:58.309 PM	192.168.1.56	3	ginomo

Figure 14: Broad first pass: every failed logon (EventID 4625), all logon types together.

`SecurityEvent`

```
| where EventID == 4625
| project TimeGenerated, IPAddress, LogonType, TargetUserName
| order by TimeGenerated desc
| take 20
```

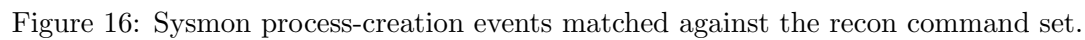
The screenshot shows a KQL query in the Microsoft Sentinel interface. The query is: `SecurityEvent | where EventID == 4624 and LogonType == 10 | project TimeGenerated, Account, IPAddress, Computer`. The results table shows 2 rows of successful RDP logons from 9/6/2026.

TimeGenerated [UTC]	Account	IPAddress	Computer
> 9/6/2026, 12:33:33.353 PM	DESKTOP-8C93D80\sylix	192.168.1.56	DESKTOP-8C93D80
> 9/6/2026, 12:33:33.353 PM	DESKTOP-8C93D80\sylix	192.168.1.56	DESKTOP-8C93D80

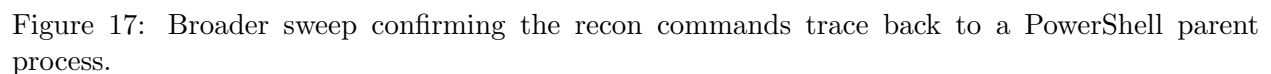
Figure 15: Successful RDP logons, isolated with `LogonType == 10` (RemoteInteractive).

`SecurityEvent`

```
| where EventID == 4624 and LogonType == 10
| project TimeGenerated, Account, IPAddress, Computer
```



```
| where Source == "Microsoft-Windows-Sysmon" and EventID == 1
| where EventData has_any ("whoami", "systeminfo", "net user", "net localgroup")
| parse EventData with * "CommandLine" * ">" CommandLine "<" *
| project TimeGenerated, Computer, CommandLine
```



My first instinct was to alert on raw `EventID == 4625` volume. That is too noisy – it mixes console logon failures, network-share auth noise, and RDP brute-force attempts into one bucket, and none of the Hydra runs above ever produced a clean, obviously-malicious signature on `EventID` alone. The detail that actually matters is **LogonType**: Hydra’s RDP module generates `LogonType == 3` (Network) failures rather than `LogonType == 10` (RemoteInteractive), because the RDP session itself never fully negotiates before the credential check fails. Filtering to `LogonType == 3` specifically is what turns a noisy log into a usable brute-force indicator.

```
| where EventID == 4625 and LogonType == 3
| summarize FailedAttempts = count() by IpAddress, Computer, bin(TimeGenerated, 1m)
| where FailedAttempts >= 5
```

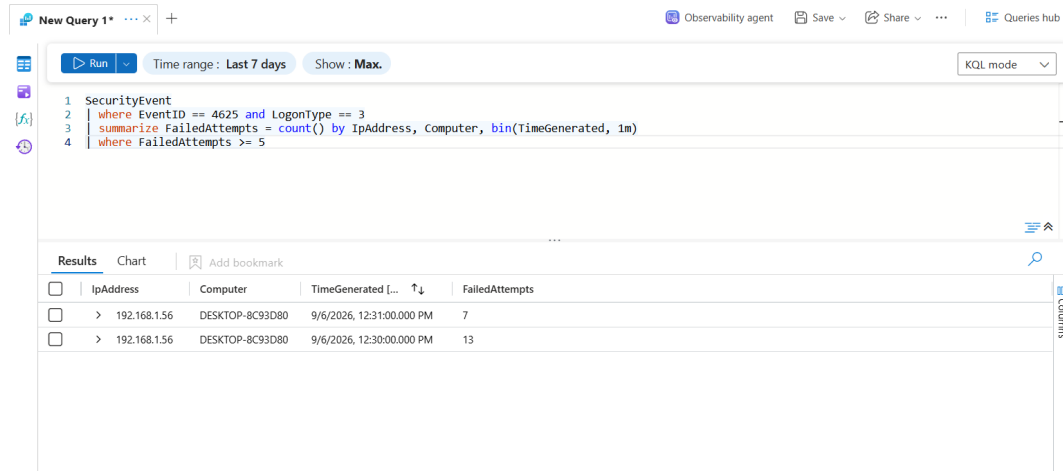


Figure 18: The corrected query – LogonType == 3, bucketed into 1-minute windows, thresholded at 5+ failures.

This is the query that became the scheduled Analytics Rule in the next section.

6 Analytics Rule and Triggered Incident

The corrected hunting query was turned into a scheduled rule, *Possible RDP Brute Force – Multiple Failed Logons*: High severity, running every 5 minutes over the last 5 minutes of data, mapped to MITRE ATT&CK Credential Access (T1110, Brute Force) and Lateral Movement (T1021, Remote Services).

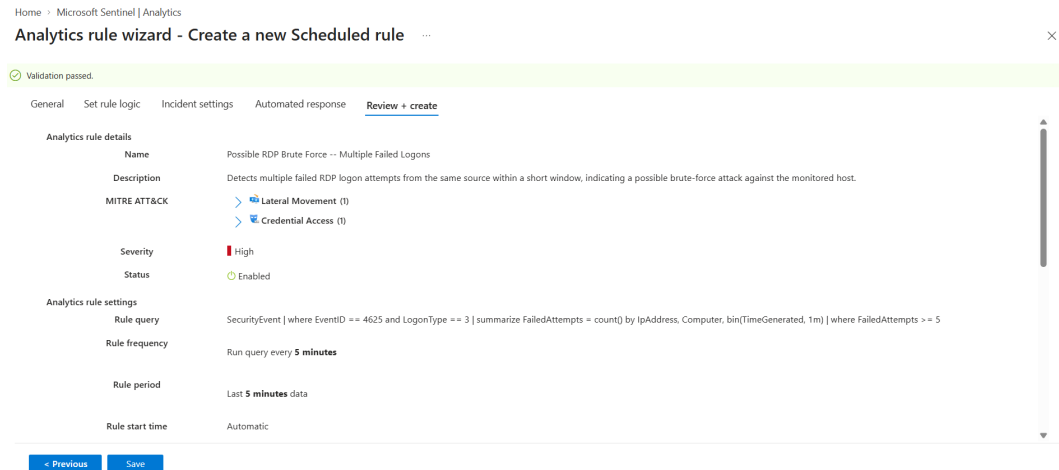


Figure 19: Analytics Rule configuration – Review + Create.

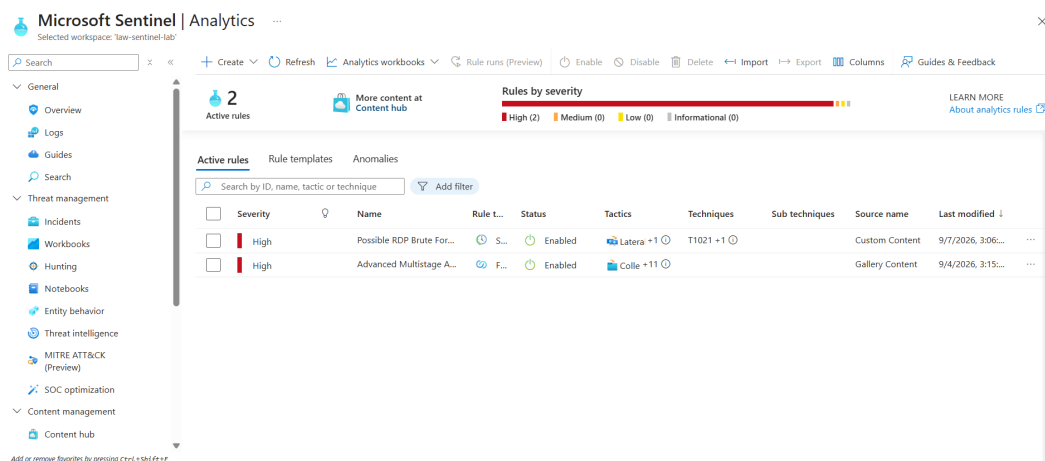


Figure 20: Active Analytics Rules in the workspace.

The rule fired for real once the attack pattern matched: the targeted successful login (Section 3.2) sitting on top of a burst of `LogonType == 3` failures was enough to cross the 5-per-minute threshold and raise *Possible RDP Brute Force – Multiple Failed Logons* as a High-severity incident.

Finding: the rule caught exactly what it was designed for, nothing more

This wasn't a scripted demo where I fired a known-good payload at a rule I'd already tuned to it – the rule was built from the hunting query in Section 4, before I knew for certain the next attack run would trigger it. It did, on the first real attempt after the rule went live, which is a reasonable (if small-sample) confirmation that the `LogonType == 3` / 5-per-minute threshold

generalizes beyond the specific data I built it on.

7 Incident Investigation

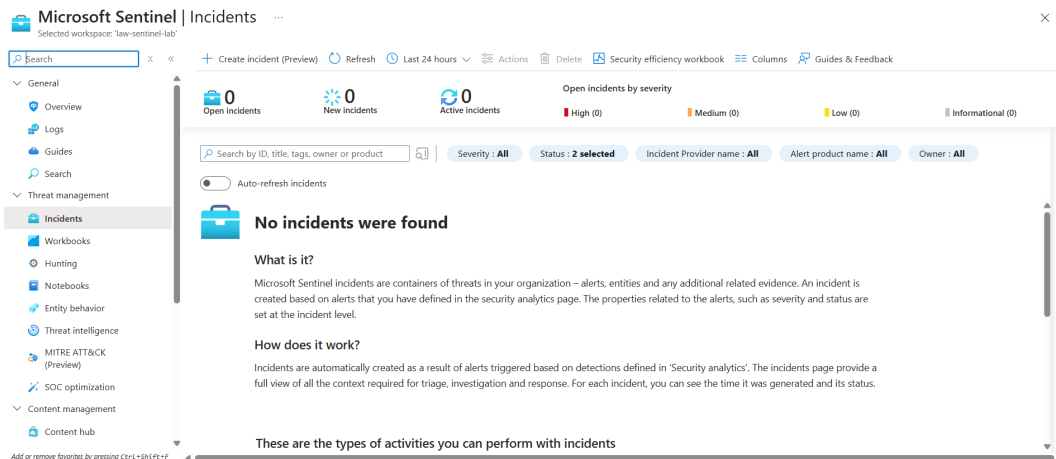


Figure 21: Incidents view before the triggering run – nothing yet.

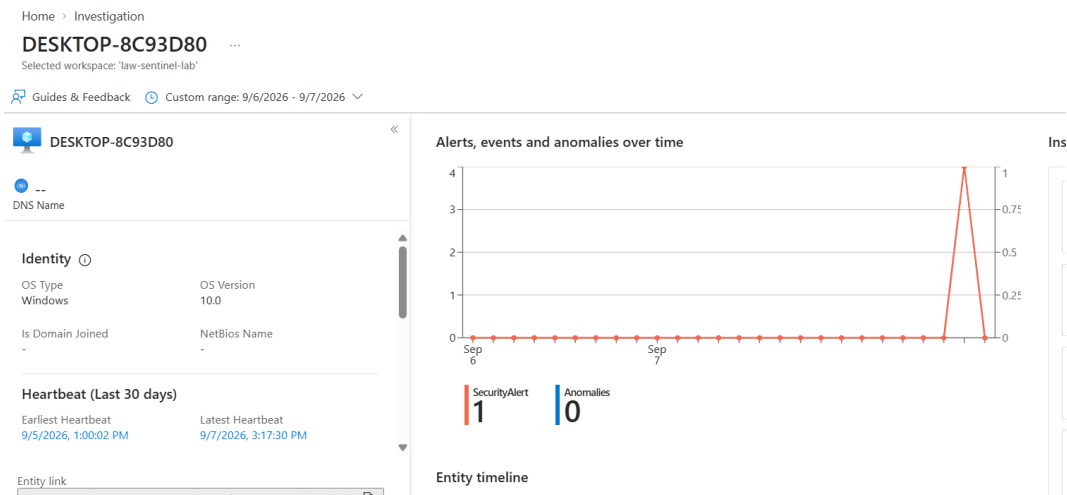


Figure 22: Entity investigation timeline for DESKTOP-8C93D80 – a single sharp alert spike against an otherwise flat multi-day baseline.

Finding: the graph confirms the hunt, it doesn't replace it

The investigation graph is a genuinely useful sanity check – it shows the same host and source IP that the manual KQL hunt already surfaced, with nothing unexpected pulled in. But it only exists because the hunting and the rule logic were right beforehand; the graph itself doesn't do any analysis, it visualizes the entities the rule already flagged. The real detection work happened in Sections 4 and 5.

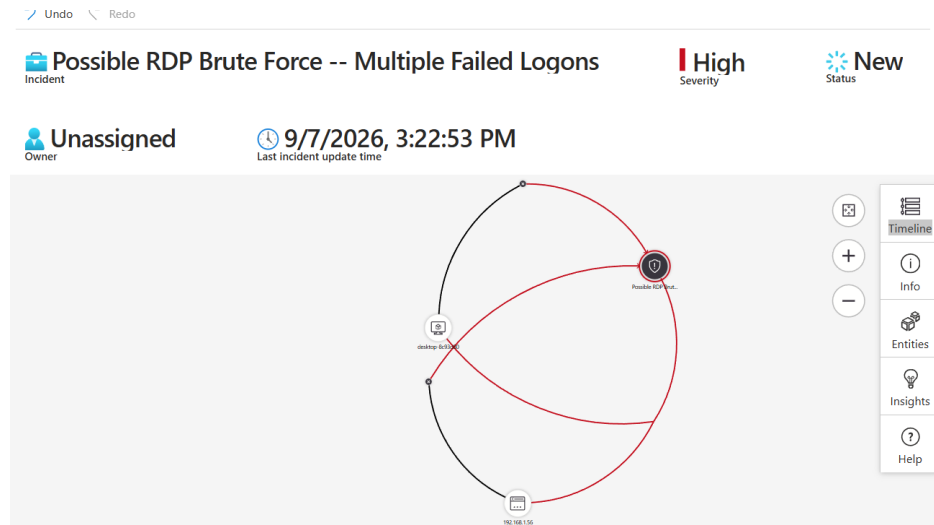


Figure 23: Investigation graph for *Possible RDP Brute Force – Multiple Failed Logons*: the alert node connects directly to the target host and the source IP – exactly the two entities the manual hunting queries in Section 4 had already pointed to.

8 Workbook

I built a custom Workbook, *RDP Intrusion – Detection Summary*, on top of the same KQL used for hunting, to give a one-screen view of the detection without opening the Logs blade every time.

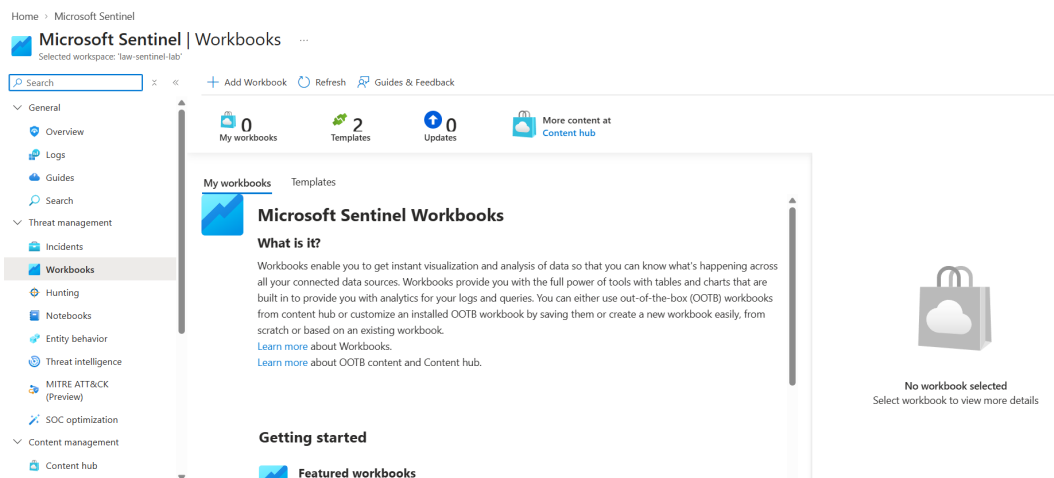


Figure 24: Sentinel Workbooks landing page before building a custom one.

Finding: the chart makes the two-session pattern obvious at a glance

The bar chart is the single most useful item in the Workbook: it visually separates the first (smaller, inconclusive) attack session from the second, larger one that actually crossed the detection threshold, without needing to re-run any query. That's the kind of at-a-glance triage view a Workbook is supposed to provide over raw log search.

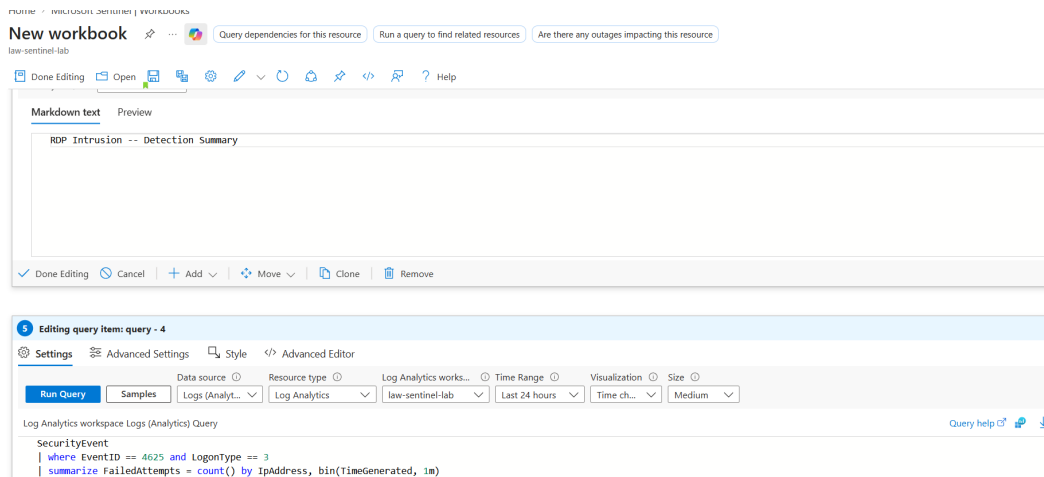


Figure 25: Workbook editor – title markdown plus the brute-force detection query as a live workbook item.

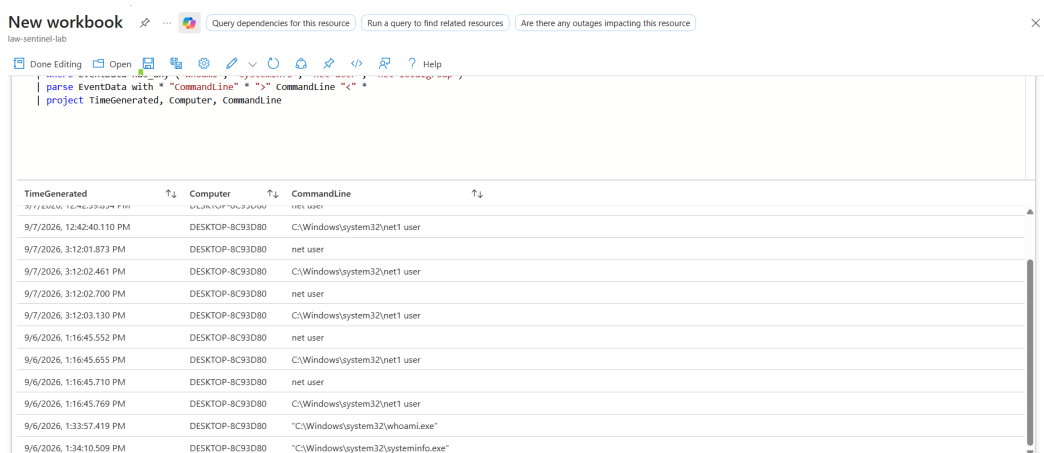
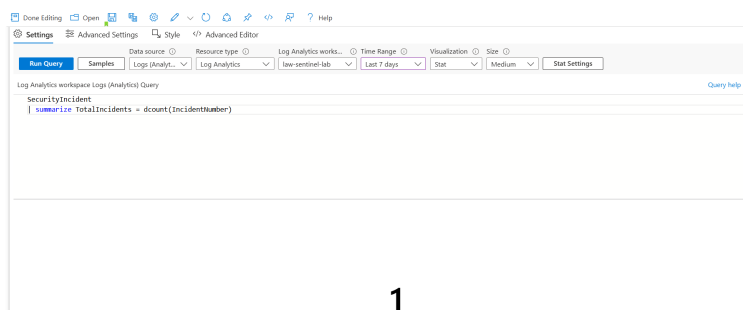


Figure 26: Recon-command table item, pulled from the Sysmon hunting query.



1

Figure 27: SecurityIncident | summarize TotalIncidents = dcount(IncidentNumber) – a one-number stat tile.

TimeGenerated	Computer	CommandLine
9/7/2026, 12:42:40.110 PM	DESKTOP-8C93DB0	C:\Windows\system32\net1 user
9/7/2026, 3:12:01.873 PM	DESKTOP-8C93DB0	net user
9/7/2026, 3:12:02.461 PM	DESKTOP-8C93DB0	C:\Windows\system32\net1 user
9/7/2026, 3:12:02.700 PM	DESKTOP-8C93DB0	net user
9/7/2026, 3:12:03.130 PM	DESKTOP-8C93DB0	C:\Windows\system32\net1 user
9/6/2026, 1:16:45.552 PM	DESKTOP-8C93DB0	net user
9/6/2026, 1:16:45.655 PM	DESKTOP-8C93DB0	C:\Windows\system32\net1 user
9/6/2026, 1:16:45.710 PM	DESKTOP-8C93DB0	net user
9/6/2026, 1:16:45.769 PM	DESKTOP-8C93DB0	C:\Windows\system32\net1 user
9/6/2026, 1:33:57.419 PM	DESKTOP-8C93DB0	"C:\Windows\system32\whoami.exe"
9/6/2026, 1:34:10.509 PM	DESKTOP-8C93DB0	"C:\Windows\system32\systeminfo.exe"

Figure 28: The recon-commands table, full width – `net user`, `whoami`, and `systeminfo` calls across both attack days.

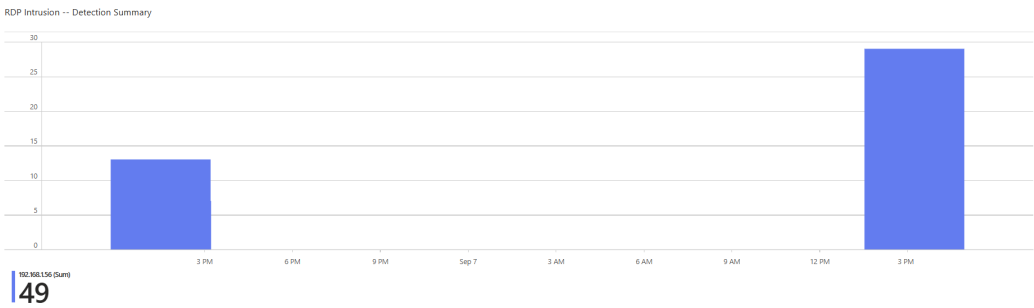


Figure 29: Failed-logon volume from the attacker IP over time – two distinct sessions: roughly 12 events on the first day, 49 total by the second.

9 MITRE ATT&CK Mapping

The Sentinel MITRE ATT&CK view maps active rules against the framework in real time.

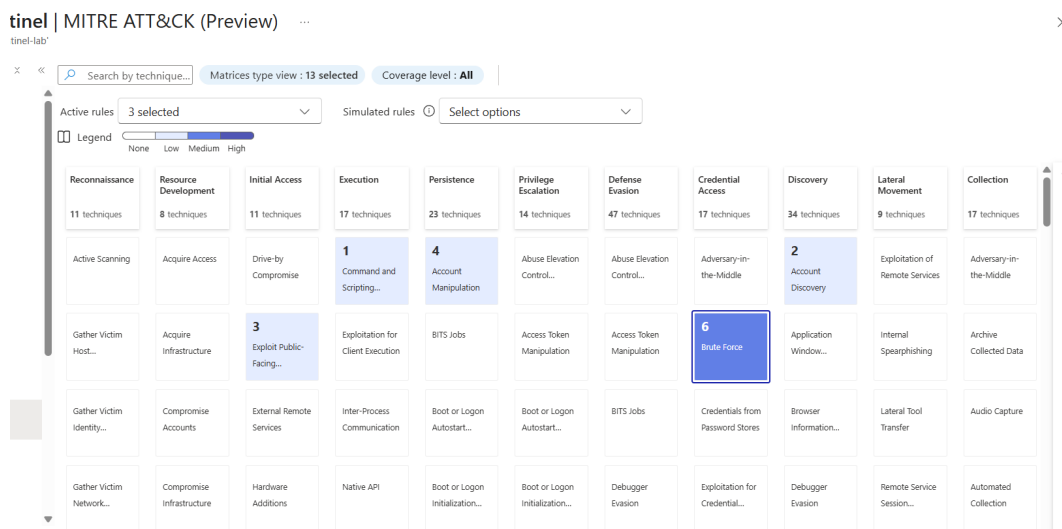


Figure 30: Brute Force (T1110) shows the highest hit count – the direct result of this lab’s own analytics rule.

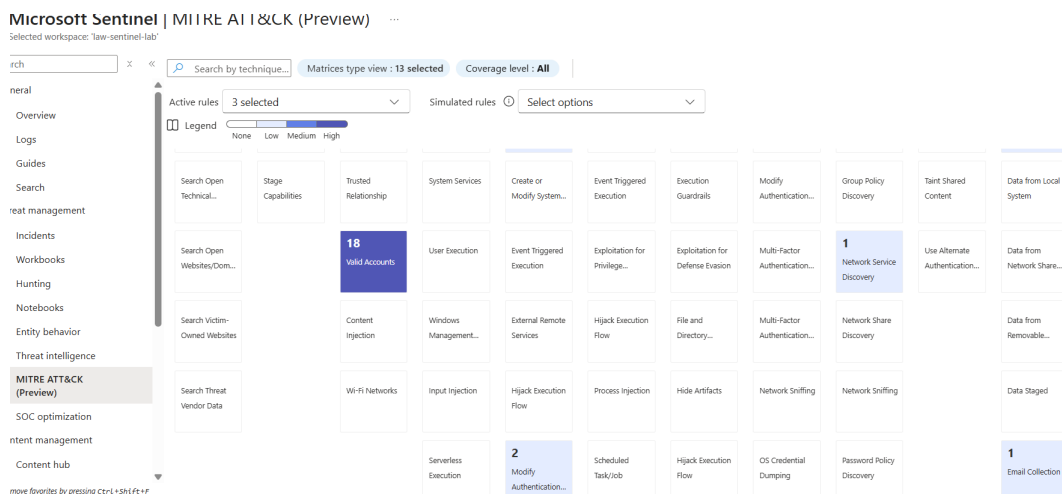


Figure 31: Valid Accounts (T1078) lights up once the targeted credential guess succeeded.

Tactic	Technique	Detected by
Discovery	T1033 / T1082 – System Owner/User & System Information Discovery	Sysmon (<code>whoami</code> , <code>systeminfo</code>)
Discovery	T1018 – Remote System Discovery	Sysmon (<code>net user</code> , <code>net localgroup</code>)
Credential Access	T1110 – Brute Force	SecurityEvent (EventID 4625, LogonType 3) – this lab's Analytics Rule
Lateral Movement	T1021.001 – Remote Services (RDP)	SecurityEvent (EventID 4624, LogonType 10)
Initial Access	T1078 – Valid Accounts	SecurityEvent (successful logon following the brute-force burst)

Finding: not every highlighted technique in the view is this lab's own

The MITRE view also lights up techniques like Command and Scripting Interpreter, Account Discovery, Account Manipulation, and Exploit Public-Facing Application – these come from the rest of the content-hub / template rule set active in the workspace, not from anything this lab specifically detected. Worth stating plainly rather than implying broader coverage than what was actually built here.

10 Automation – SOAR Playbook

To close the loop, I built a Logic App playbook, `pb-rdp-bruteforce-notify2`, and wired it to the analytics rule with an Automation Rule.

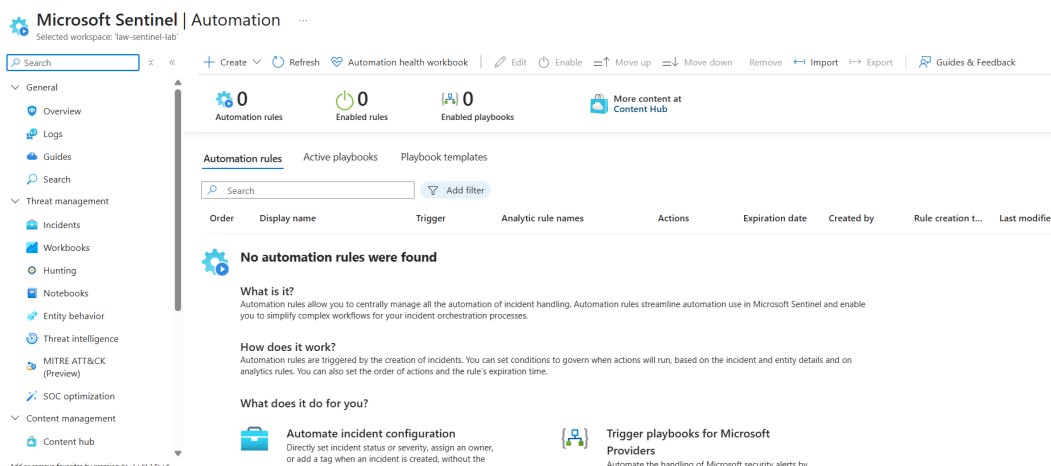


Figure 32: Automation page before any rule or playbook existed – zero automation rules, zero enabled playbooks.

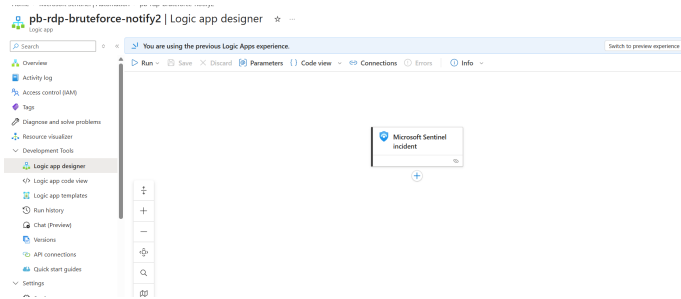


Figure 33: Logic App designer – *Microsoft Sentinel incident* trigger added.

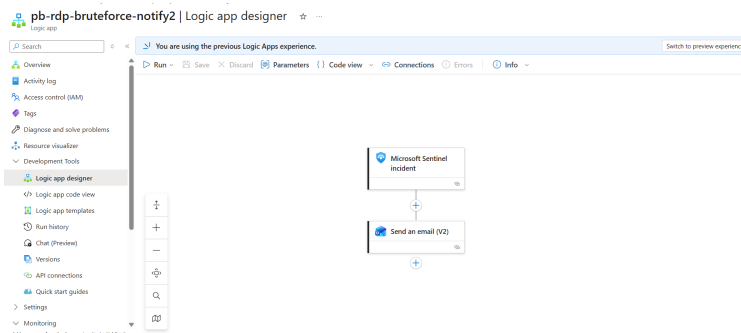


Figure 34: *Send an email (V2)* action wired directly after the incident trigger.

Finding: a detection nobody gets notified about isn't operational

The Analytics Rule and the KQL behind it were the hard part, but they were only ever going to matter if someone actually finds out when they fire. Wiring the playbook in is what turns “a query that would catch it” into “a detection that tells someone when it does.” It’s a deliberately small first step – a single email action – but the same Automation Rule pattern scales to isolating a host, disabling an account, or opening a ticket without changing the trigger logic.

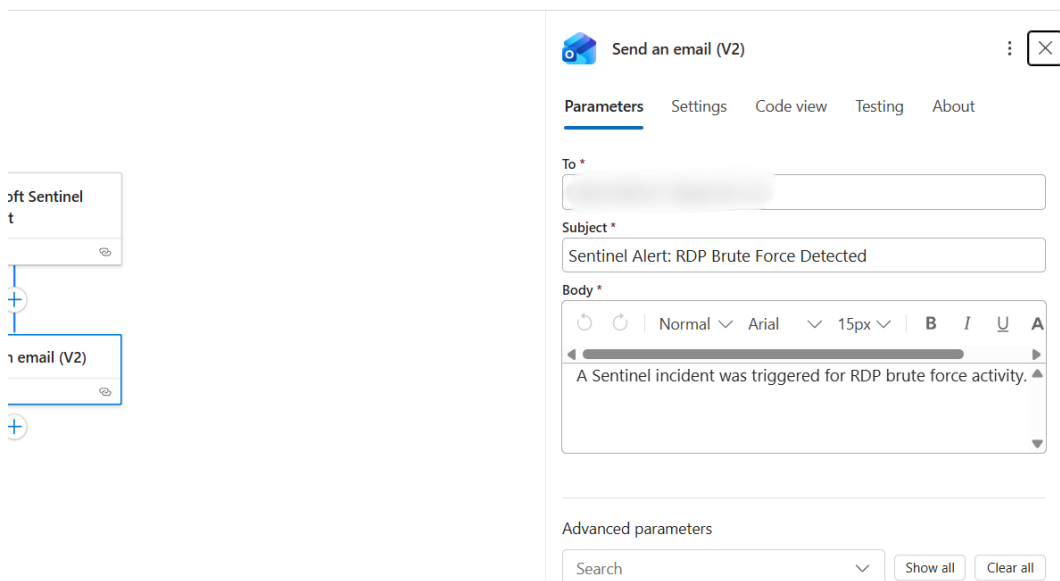


Figure 35: Email action configuration – subject “Sentinel Alert: RDP Brute Force Detected”.

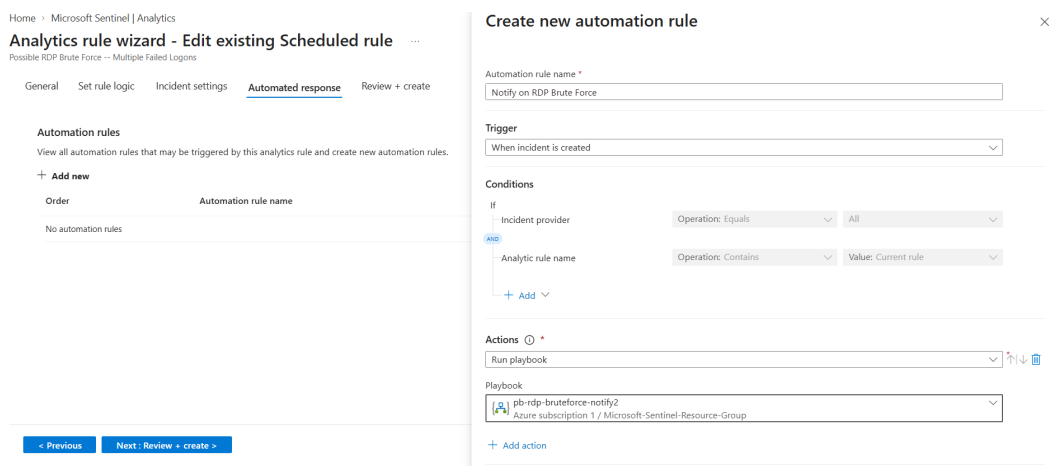


Figure 36: Automation Rule *Notify on RDP Brute Force* – fires “When incident is created”, condition on this specific analytics rule name, action *Run playbook*.

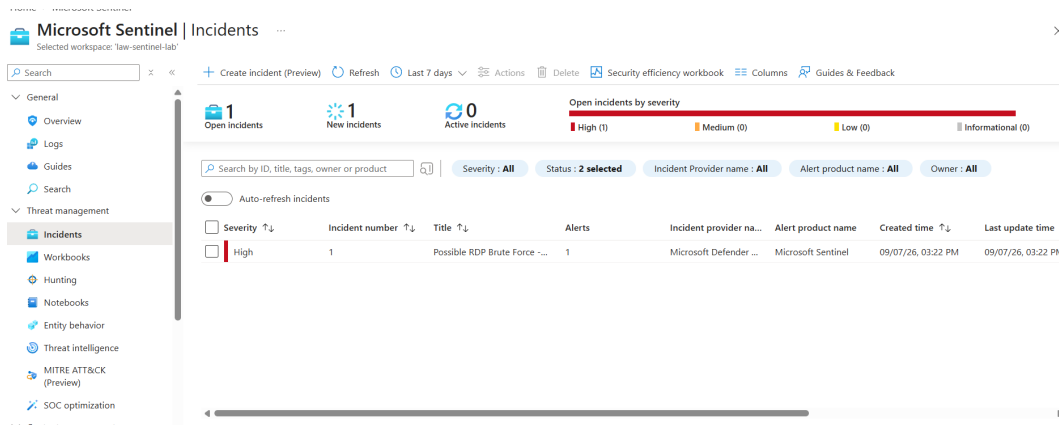


Figure 37: Incidents list showing the open High-severity incident after the automation rule fired.

11 Lessons Learned

- **Query filtering has to be deliberate, or it's just noise.** The biggest thing this lab reinforced for me is that you always have to filter your queries carefully – alerting on a raw `EventID == 4625` count would have been too noisy to act on. Narrowing it down to the specific `LogonType` that actually matches the attack behavior (`LogonType == 3` for Hydra's RDP module, not `LogonType == 10`) is what turned a noisy log into something I could actually threshold and alert on.
- **Manual hunting before automation builds real confidence.** Writing the KQL by hand against actual attack data, before any scheduled rule existed, meant the eventual threshold (5+ failures/minute) came from what the attack really produced instead of a guess – and it held up on the very next real attack run.
- **The MITRE ATT&CK view can overstate coverage if you're not careful.** It's easy to glance at a lit-up matrix and assume broad detection coverage. Several of the techniques highlighted in this workspace's view came from content-hub template rules, not from anything this lab actually built, and it's worth being explicit about that distinction rather than letting the visualization imply more than it should.

12 Recommendations

1. **Move past notification-only automation.** Right now the playbook only sends an email when the incident fires. The next step is wiring in a real response action – disabling the compromised account or isolating the host – instead of just notifying, once I'm confident enough in the detection's accuracy to let it act automatically.
2. **Establish a false-positive baseline.** The 5-per-minute threshold was tuned against Hydra's attack traffic specifically; it hasn't been validated against benign RDP failures (mistyped passwords, expired sessions, a legitimate user locked out). Testing the rule against normal day-to-day noise would be the natural next step before trusting it in anything closer to a real environment.

With this lab and the RDP and AWS IAM labs behind me, my next goal is preparing for the Security Blue Team Level 1 (BTL1) certification, to formalize the SOC analyst skills these projects have been building hands-on.